# DL26, Assignment 2

Anh Tu Duong Nguyen (anguyea, 2115931)
Anh-Nhat Nguyen (anhnnguy, 2034311)

April 25, 2026

## Task 2a)

MNIST-1D comprises ten templates; each is a 12-point 1D signal representing a digit (0–9). The script whitens these templates to zero mean and unit variance. Each instance is created by padding the signal to length 40, then applying stochastic transformations: random translation, scaling, and shearing. Finally, the process injects correlated and independent noise, forcing models to learn features invariant to spatial shifts and distortion.

## Task 2c)

For a $n \times m$ kernel with $c_{in}$ input channels and $c_{out}$ output channels we have: $n \cdot m \cdot c_{in} \cdot c_{out}$ parameters. However, since we are dealing with one-dimensional data here, we can set $n = 1$. Therefore, the formula becomes $m \cdot c_{in} \cdot c_{out}$. The pairs of input and output channels are given as follows:

- $(1,\ 5)$
- $(5,\ 5)$

In the Linear Layer at the end of this CNN, the number of parameters is given by the weight matrix and bias vector where the height of the weight matrix is given by the number of output channels of the last convolutional layer. In this case, it is $(5,\ 10)$ as 10 is the number of classes in our dataset. Furthermore, we also need to account for the bias terms per layer which equals the number of outputs a layer has. For the conv layers we have 5 and for the linear one we have 10 bias terms.

- channels $= 5$
- kernel size $= 3$
- stride $= 2$
- padding $= 1$

So for every layer we have the following number of parameters:

- $3 \cdot 1 \cdot 5 = 15$
- $3 \cdot 5 \cdot 5 = 75 \ \rightarrow$ occurs two additional times
- $5 \cdot 10 = 50$

Adding the bias terms, we have:

- $15 + 5 = 20$
- $75 + 5 = 80 \ \rightarrow$ occurs two additional times
- $50 + 10 = 60$

To get the total number of parameters in the CNN or in any neural network, we need to sum up the number of parameters in each layer. In total, we have $20 + 80 \cdot 3 + 60 = 320$ parameters in this CNN.

## Task 2d): i), ii), and iii)

i) We have a finite dataset so it is better to use map-style datasets as we are not dealing with a continuous stream of data. Furthermore, random access is therefore possible and beneficial in such a dataset for several reasons:

- Shuffling is possible → preventing the model from learning the sequence order
- Subset splitting → trivial train/test split

On the other hand, iterable-style datasets are more suitable for streaming data or when the dataset is too large to fit into memory, which is not the case here.

ii) According to definition of a Map-style dataset we need an implementation of the `__getitem__` and `__len__` function. `TensorDataset` provides such an implementation of those functions to allow random reads via indices so that a data point and its label can be extracted from it while also getting the size of the dataset through this implementation.

iii) `CrossEntropyLoss` and `NLLLoss` are in essence very similar and are very useful for the same use cases, such as classification with $C$ classes. However, `CrossEntropyLoss` is much more flexible and does not require us to add an additional `LogSoftmax` layer at the end, but we can directly feed the output of the CNN to the `CrossEntropyLoss` function. On the other hand, `NLLLoss` requires us to add a `LogSoftmax` layer at the end of the CNN to get the log-probabilities of the output which is then fed to the `NLLLoss` function.

## Task 2e)

Following the equation from task c), we can calculate the number of parameters for the `SimpleMLP` as follows:

- $(40 \times 256) + 256 = 10496$
- $(256 \times 256) + 256 = 65792$
- $(10 \times 256) + 10 = 2570$

The number of parameters are 78858.

We conjecture that our current `SimpleCNN` will outperform the `SimpleMLP` in terms of test performance since we have a much smaller number of parameters to fit which reduces the risk of overfitting. We know from the lecture that the MLP may be able to fit any kind of function by using wider layers, but this tends to overfit.

## Task 2f)

Our conjecture was correct since from the results it can be seen that the `SimpleCNN` outperforms the Multi-Layer Perceptron. In terms of test accuracy we have the following results:

- `SimpleCNN`: 92%
- `SimpleMLP`: 63%

This displays the advantage of CNNs over MLPs in terms of handling stationary data.

## Task 2h)

We are shuffling the data across the spatial dimension meaning that the spatial information gets lost in the images since we are rearranging those pixels. From the lecture we know that this rearrangement leads to information loss and especially since we are shuffling the data randomly, no usefule patterns can be learned from the underlying data and therefore, we assume that the CNN loses to the conventional MLP, but even so the MLP still overfits due to this large number of parameters and the fact that MLPs do not make use of the spatial information in the data.

## Task 2i)

The conjecture about the performance was correct as with the randomly shuffled data information and structure got lost that normally allows CNN to learn the patterns in the data, but since our inductive bias of stationary data does not hold here anymore, the CNN model was less performant the Multi-Layer Perceptron. In detail, the test accuracy of the CNN dropped to 56.8% while the MLP still achieved a test accuracy of 63%. This displays the importance of the inductive bias since in this scenario the inductive bias was violated and therefore training failed for the CNN.

## Task 3c)

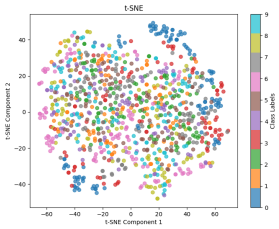
(i) Input data. The projection is expected to show overlapping classes with no clear cluster structure, since similarity in signal amplitude does not align with digit labels.

(ii) Training embeddings. Training samples are expected to be mapped to a space with strong class separation. The projection is expected to show compact and well-separated clusters.

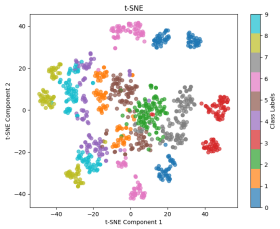
(iii) Test embeddings. The projection is expected to show identifiable clusters with greater dispersion, along with points near class boundaries and some misclustered samples.

## Task 3d)

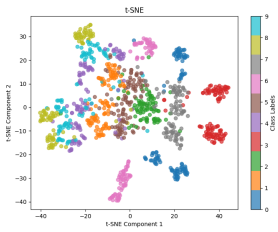
The t-SNE projections show how the CNN arranges samples in the embedding space.




(a) Raw Embeddings




(b) Train Embeddings




(c) Test Embeddings

**Training embeddings**:
The model reaches 99.8% training accuracy with a loss of $1.168 \times 10^{-3}$. The embeddings form tight, well-separated clusters, which indicates clear class separation.

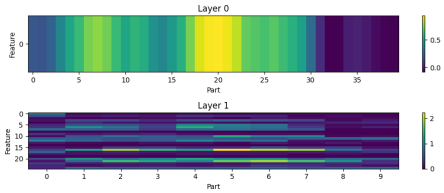
**Test Embeddings**:
The test projection contains distinct clusters with higher spread. Some points lie near class boundaries or overlap with neighboring clusters. Test accuracy is 91.2%, and the loss increases to 0.538, which shows lower confidence on unseen data.

**Interpretation**:
These results show a generalization gap. Training embeddings are compact because the model fits the training data closely. Test embeddings spread toward class boundaries, which reduces confidence. Accuracy remains high because most predictions are correct, but the higher loss shows that predicted probabilities are less certain for some samples.

**t-SNE parameter selection**:
We set the perplexity to 30 to balance local and global structure. Lower values split classes into small clusters, while higher values merge classes and blur boundaries, especially in the test embeddings (loss 0.538). At 30, clusters remain stable and separated, with controlled spread in the test data, which supports consistent comparison.

## Task 3e)

We apply t-SNE to embeddings from the first convolutional block (conv-block 1) to examine early representations.




(a) t-SNE projection of first conv-block embeddings

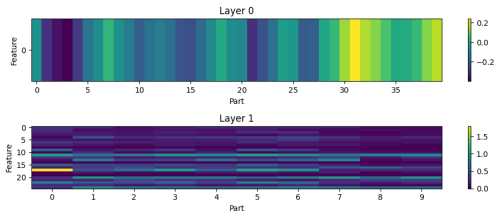
The projection shows substantial overlap between classes and weak cluster boundaries. In contrast, embeddings from the final layer form compact and well-separated clusters.

This difference reflects the hierarchical structure of CNNs. Early layers detect local patterns such as edges and gradients that appear across many digit classes. As a result, samples from different classes remain close in the embedding space. Deeper layers combine these patterns into more specific features, which increases separation between classes.

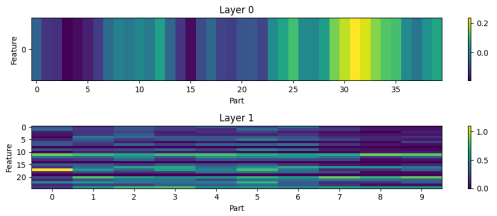
The comparison shows that class separation improves with depth: early embeddings mix classes, while final-layer embeddings organize samples into distinct groups.

## Task 3f)

We compare activation heatmaps from the first convolutional layer for classes 1, and 5 to examine how kernels respond to different inputs.




(a) t-SNE projection of Class 1




(a) t-SNE projection of class 5

**Class-specific activation:** Different kernels respond to different classes. For example, feature 17 in the first layer produces high activation for class 1, which contains a vertical stroke, and low activation for class 0, which is dominated by curved structure. This result shows that early-layer kernels detect specific shapes.

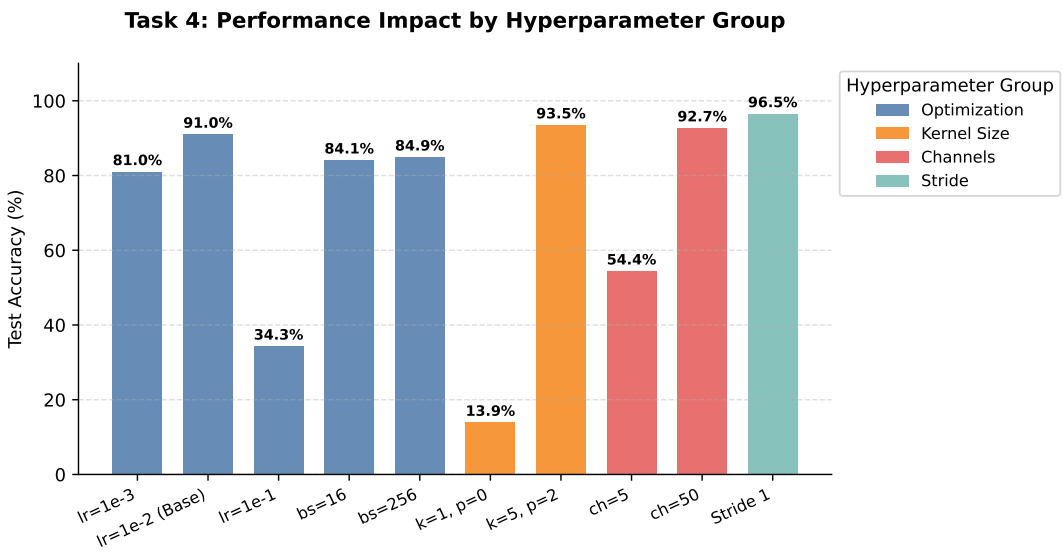
**Spatial sensitivity:** Activation depends on position. For class 0, peak responses occur near the center (positions 15–25). For class 1, responses shift to the right (positions 30–40). For class 5, which combines straight and curved segments, high responses appear at multiple positions where these structures occur. Kernel responses therefore depend on both feature type and location.

**Conclusion:** The heatmaps show that each kernel detects a limited set of patterns at specific locations. Classification depends on the combined responses across kernels. This combination separates digits with similar structure, such as 0, 1, and 5.

## Task 4)

We evaluated hyperparameter impacts on MNIST-1D performance, comparing configurations for optimization, geometry, width, and stride against a baseline ($lr = 10^{-2}, bs = 64, k = 3, ch = 25, stride = 2$, 6060 parameters).

| Group | Setting | Params | Acc. | Group | Setting | Params | Acc. |
|---|---|---|---|---|---|---|---|
| Optim | $lr = 10^{-3}$ | 6060 | 81.0% | Geom | $k = 1$ (1x1) | 2260 | 13.9% |
| Optim | **Base** ($10^{-2}$) | 6060 | 91.0% | Geom | $k = 5, p = 2$ | 9860 | 93.5% |
| Optim | $lr = 10^{-1}$ | 6060 | 34.3% | Width | $ch = 5$ | 320 | 54.4% |
| Optim | $bs = 16$ | 6060 | 84.1% | Width | $ch = 50$ | 23360 | 92.7% |
| Optim | $bs = 256$ | 6060 | 84.9% | Stride | **Stride 1** | 8310 | **96.5%** |




**Discussion:** We found that the baseline parameters ($lr = 10^{-2}$) were optimal for stable convergence. Larger kernels ($k = 5$) improved performance over the baseline by increasing the receptive field, while increasing width ($ch = 50$) offered diminishing returns. The most substantial accuracy gain resulted from reducing the stride from 2 to 1 (96.5%). This change demonstrates that preserving spatial resolution throughout the network is critical, as aggressive early downsampling discarded essential structural features.

## Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

**Declaration of Used AI Tools**

| Task | Solved with AI? | Tool | Purpose | Useful? |
|---|---|---|---|---|
| 1b | Yes | ChatGPT | Generate draft solution | yes |
| 1c | | | | |
| 1d | | | | |
| 2a | | | | |
| 2b | | | | |
| 2c | | | | |
| 2d | | | | |
| 3a | | | | |
| 3b | | | | |

Unterschrift
Mannheim, den XX.XX.2025